

1. Define a Transaction. Explain the ACID Properties of Transactions with Suitable Examples.

Transaction:

A transaction is a set of logically related operations that are executed as a single unit of work.

An application program may contain several transactions separated by Begin Transaction and End Transaction boundaries.

The basic operations of a transaction are:

- **Read_item(X):** Reads a database item X into a program variable.
- **Write_item(X):** Writes the value of the program variable back into the database item X.

Read Operation:

The read_item(X) operation consists of the following steps:

1. Find the address of the disk block containing item X.
2. Copy the disk block into the main memory buffer (if it is not already present).
3. Copy item X from the buffer to the program variable.

Write Operation:

The write_item(X) operation consists of the following steps:

1. Find the address of the disk block containing item X.
2. Copy the disk block into the main memory buffer.
3. Copy the updated value from the program variable into the buffer.
4. Store the updated block back to disk.

Example of a Transaction:

A simple bank money transfer transaction:

1. Read(A)
2. $A = A - 10000$
3. Write(A)
4. Read(B)
5. $B = B + 10000$
6. Write(B)

Commit

Here,

- Amount ₹10,000 is deducted from Account A.
- Amount ₹10,000 is added to Account B.
- The transaction is completed only after Commit.

ACID Properties of Transactions:

The ACID properties ensure the integrity, consistency, and reliability of the database during transaction processing.

ACID stands for:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

1. Atomicity:

Atomicity ensures that either all the operations of a transaction are executed successfully or none of them are executed.

A transaction should never remain partially completed.

Example-

Bank Transfer

Account A = ₹500

↓

Deduct ₹100

↓

Account B = ₹300

If the amount is deducted from A but not added to B due to a system failure, the transaction is rolled back.

Hence,

Either both operations occur or neither occurs.

Importance of Atomicity

- Prevents partial updates.
- Protects database integrity.

- Avoids data corruption during failures.

If atomicity is not ensured:

- Money may be deducted but not credited.
- Partial updates remain in the database.
- Database becomes inconsistent.

2. Consistency:

Consistency ensures that the database remains correct and valid before and after every transaction.

All integrity constraints must be satisfied.

Example:

Before transaction

Account A = 500

Account B = 200

Total = 700

After transfer

Account A = 400

Account B = 300

Total = 700

Since the total balance remains unchanged, the database is consistent.

If only deduction occurs and addition fails,

$400 + 200 = 600$

Database becomes inconsistent.

3. Isolation:

Isolation ensures that multiple transactions execute independently without interfering with one another.

The changes made by one transaction are not visible to another transaction until they are committed.

Example:

Suppose $X = 500$ $Y = 500$

Transaction T1 updates the values.

Before T1 completes, another transaction T2 reads X and Y.

T2 may read

$X = 50000$

$Y = 500$

and compute

$X + Y = 50500$

whereas after T1 completes,

$X + Y = 50450$

This leads to inconsistency.

Therefore, transactions must execute in isolation.

4. Durability:

Durability ensures that once a transaction is committed, its changes become permanent.

Even if a system failure occurs after commit, the updated data is not lost.

Example:

If a customer deposits ₹5,000 and the transaction is committed,

Even if the system crashes immediately afterward,

the deposited amount remains stored permanently in the database.

Difference Between Atomicity and Consistency

Atomicity	Consistency
Ensures all operations execute completely or none execute.	Ensures database remains valid before and after transaction.
Prevents partial execution.	Prevents violation of integrity constraints.
Concerned with transaction completion.	Concerned with correctness of data.
Example: Money is either both deducted and credited or neither.	Example: Total balance before and after transaction remains the same.

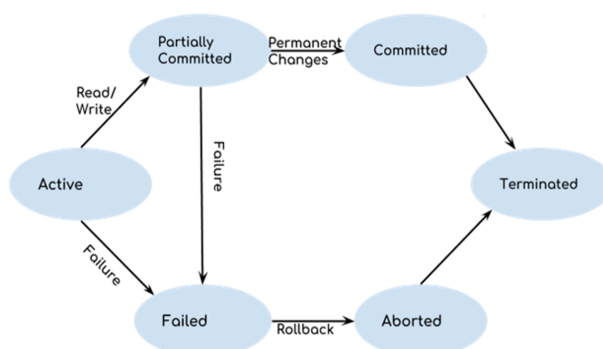
Advantages of ACID Properties

- Maintains data integrity.
- Ensures reliable transaction processing.
- Prevents data inconsistency.
- Supports concurrent transaction execution.
- Guarantees permanent storage of committed data.

2.Explain the Different Transaction States with a Neat State Diagram. Also Explain the Causes of Transaction Failures.

A transaction passes through different states during its execution. These states indicate the progress of a transaction from the beginning until it is either successfully completed or aborted.

Transaction State Diagram:



1. Active State:

A transaction is said to be in the Active State when it is currently executing its read and write operations.

Characteristics-

- Transaction execution has started.
- Read and write operations are performed.
- Transaction may proceed successfully or fail due to an error.

Example:

Read(A)

A = A – 500

Write(A)

The transaction is in the Active State while these operations are being executed.

2. Partially Committed State:

A transaction enters the Partially Committed State after executing all its operations but before the changes are permanently stored in the database.

This state helps the DBMS roll back the transaction if a failure occurs before committing.

Characteristics-

- All read and write operations are completed.
- Changes are still in local memory/buffer.
- Transaction is not yet permanently saved.

Example:

A bank transfer transaction has completed all operations, but before the commit is written to disk, a power failure occurs.

The transaction can still be rolled back.

3. Committed State:

A transaction enters the Committed State after successful completion of all operations and permanent storage of all changes in the database.

Characteristics-

- Transaction executes successfully.
- Updates become permanent.
- Changes cannot be undone under normal conditions.

Example:

After transferring ₹10,000 from Account A to Account B, both balances are permanently stored in the database.

The transaction is now committed.

4. Failed State:

If an error occurs while executing a transaction, it enters the Failed State.

The failure may occur due to hardware, software, or logical errors.

Characteristics-

- Transaction cannot continue.
- Recovery process starts.
- Transaction will be rolled back.

Example:

While updating the database, a system crash occurs before the transaction completes.

The transaction moves to the Failed State.

5. Aborted State:

When a failed transaction is rolled back and all its changes are undone, it enters the Aborted State.

Characteristics-

- All updates made by the transaction are cancelled.
- Database returns to its previous consistent state.
- The transaction may either restart or terminate permanently.

Example:

If money is deducted from one account but cannot be credited to another due to failure, the deducted amount is restored through rollback and the transaction is aborted.

Difference Between Partially Committed and Committed States:

Partially Committed	Committed
Transaction has finished execution but changes are not yet permanent.	Transaction has successfully completed and changes are permanently stored.
Rollback is still possible if failure occurs.	Rollback is generally not required after commit.
Data is stored in temporary/local memory.	Data is permanently written to the database.

What Happens When a Transaction Goes to the Failed State?

When a transaction enters the Failed State:

- Execution stops immediately.
- Recovery Manager initiates the rollback process.
- All changes made by the transaction are undone.
- Database is restored to its previous consistent state.
- The transaction enters the Aborted State and may be restarted later.

Causes of Transaction Failures:

A transaction may fail due to the following reasons:

1. Computer Failure (System Crash)-

- Hardware, software, or network failure may interrupt an executing transaction.

2. Transaction Errors-

- User aborts the transaction.
- Logical programming errors.
- Arithmetic errors such as division by zero.

3. External Causes / Exception Conditions-

- Required data is not found.
- Unexpected exceptions occur during execution.

4. Concurrency Control Enforcement-

- Deadlocks.
- Violation of serializability.
- DBMS aborts the transaction to maintain consistency.

5. Disk Failure-

- Read/write malfunction.
- Disk crash causing data loss.

6. Physical Problems and Catastrophes-

- Power failure.
- Fire.
- Theft.
- Sabotage.
- Air-conditioning failure.
- Accidental overwriting of storage media.

Summary of Transaction States:

State	Description
Active	Transaction is executing.
Partially Committed	Execution completed, waiting for permanent storage.
Committed	Transaction completed successfully; changes are permanent.
Failed	Transaction cannot continue due to an error.
Aborted	Transaction is rolled back and all changes are undone.

3.What is Concurrency Control? Explain the Problems Caused by Concurrent Execution of Transactions with Suitable Examples.

Concurrency Control:

Concurrency Control is the process of managing the simultaneous execution of multiple transactions in a database so that the database always remains in a consistent state.

In DBMS, multiple transactions are executed concurrently to improve system performance. However, if they are not properly controlled, inconsistencies may occur. Therefore, concurrency control ensures correct and reliable execution of transactions.

Concurrency control is required to:

- Maintain database consistency.

- Prevent data inconsistency during concurrent execution.
- Improve CPU and disk utilization.
- Increase transaction throughput.
- Reduce response time and waiting time.
- Allow multiple users to access the database simultaneously without conflicts.

Advantages of Concurrent Execution:

1. **Increased Processor and Disk Utilization**
 - One transaction can use the CPU while another performs disk I/O.
2. **Higher Transaction Throughput**
 - More transactions are completed in less time.
3. **Reduced Response Time**
 - Short transactions need not wait for long transactions to finish.

Problems Due to Concurrent Transactions-

Concurrent execution may lead to the following problems:

1. **Lost Update Problem**
2. **Dirty Read Problem (Temporary Update Problem)**
3. **Unrepeatable Read Problem**
4. **Phantom Read Problem**

1. Lost Update Problem (Write–Write Conflict):

The **Lost Update Problem** occurs when two transactions update the same data item simultaneously, causing one update to overwrite the other.

Example-

Suppose the initial account balance is:

A = ₹1000

Transaction T1

Read(A)

A = A + 500

Write(A)

Transaction T2

Read(A)

A = A - 200

Write(A)

If both transactions read ₹1000 at the same time:

- T1 writes ₹1500
- T2 later writes ₹800

The update made by T1 is overwritten by T2.

Final Balance = ₹800

Correct balance should be:

$1000 + 500 - 200 = ₹1300$

Thus, one update is lost, leading to incorrect data.

2. Dirty Read Problem (Temporary Update Problem) (Write-Read Conflict):

The Dirty Read Problem occurs when one transaction reads data that has been updated by another transaction before it is committed.

If the first transaction later fails, the second transaction has already used an invalid value.

Example:

Transaction T1

Read(A)

$A = A + 500$

Write(A)

Before T1 commits,

Transaction T2 reads the updated value of A.

Later, T1 fails and rolls back.

Now T2 has used a value that never actually became permanent.

This is called a Dirty Read because T2 reads uncommitted (temporary) data.

3. Unrepeatable Read Problem (Read-Write Conflict):

An Unrepeatable Read occurs when a transaction reads the same data item twice and another transaction modifies that data between the two reads.

As a result, the first transaction gets different values for the same data item.

Example

Time	Transaction T1	Transaction T2
t1	Read(A)	
t2		Read(A)
t3		$A = A + 30$
t4		Write(A)
t5	Read(A)	

The first read and second read performed by T1 return different values because T2 updated A in between.

Hence, the read operation is not repeatable.

4. Phantom Read Problem:

The Phantom Read Problem occurs when a transaction reads a data item, and during a later read, the same data item is no longer present because another transaction has deleted it.

Example:

Transaction T1

Read(X)

Transaction T2

Read(X)

Delete(X)

Commit

Transaction T1 again

Read(X)

Now T1 cannot find X because it has been deleted by T2.

This unexpected disappearance of data is called the Phantom Read Problem.

Summary of Concurrency Problems

Problem	Conflict Type	Description
Lost Update	Write–Write (W–W)	One transaction overwrites another transaction's update.
Dirty Read	Write–Read (W–R)	Reads uncommitted data written by another transaction.
Unrepeatable Read	Read–Write (R–W)	Same data item gives different values on repeated reads.
Phantom Read	Read/Delete	A previously existing data item disappears due to another transaction.

How Concurrency Control Solves These Problems:

Concurrency control uses techniques such as:

- Locking mechanisms
- Serializability
- Timestamp Ordering
- Two-Phase Locking (2PL)
- Validation-Based (Optimistic) Protocols

These techniques ensure that concurrent transactions execute correctly while maintaining database consistency.

4.Explain Schedules in DBMS. Differentiate Between Serial and Non-Serial Schedules. What is Serializability? Explain Conflict Serializability and View Serializability.

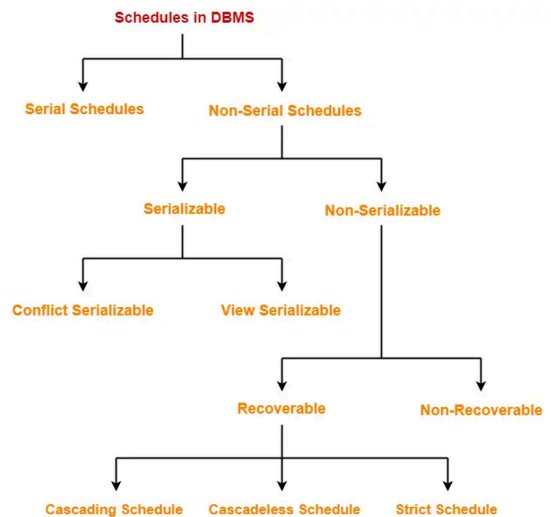
Schedules in DBMS:

A Schedule is a sequence of instructions that specifies the chronological order in which the operations of concurrent transactions are executed.

A schedule:

- Contains all operations of all transactions.
- Preserves the order of operations within each transaction.
- Ends with Commit if successful or Abort if unsuccessful.

Types of Schedules-



There are two types of schedules:

1. **Serial Schedule**
2. **Non-Serial Schedule**

1. Serial Schedule:

A Serial Schedule is one in which one transaction completes completely before another transaction starts.

There is no interleaving of operations.

Characteristics-

- Only one transaction executes at a time.
- Degree of concurrency = 1
- Database always remains consistent.
- No concurrency.

Example

T1	T2
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(B)
	Write(B)
	Read(A)
	Write(A)

Advantages:

- Simple to execute.
- Always produces a consistent database.
- No concurrency problems.

Disadvantages:

- Poor CPU utilization.
- Higher waiting time.
- Lower throughput.

2. Non-Serial Schedule:

A Non-Serial Schedule is one in which operations of multiple transactions are interleaved.

Characteristics-

- Multiple transactions execute simultaneously.
- Provides concurrency.
- Better CPU utilization.
- May lead to inconsistency if not controlled.

Example

T1	T2
Read(A)	
	Read(B)
	Write(B)
Write(A)	
Read(B)	
	Read(A)
	Write(A)
Write(B)	

Advantages:

- Higher throughput.
- Better processor utilization.
- Reduced response time.

Disadvantages:

- May produce concurrency problems.
- Requires concurrency control mechanisms.

Difference Between Serial and Non-Serial Schedules

Serial Schedule	Non-Serial Schedule
Transactions execute one after another.	Transactions execute simultaneously.
No interleaving of operations.	Operations are interleaved.
Degree of concurrency is 1.	High degree of concurrency.
Always consistent.	May become inconsistent.
Low throughput.	High throughput.
No concurrency problems.	Concurrency problems may occur.

Serializability:

Serializability is the property that ensures a non-serial schedule produces the same result as some serial schedule.

It guarantees database consistency while allowing concurrent execution.

Importance of Serializability-

- Ensures database consistency.
- Allows safe concurrent execution.
- Prevents incorrect results.
- Improves performance without sacrificing correctness.

A non-serial schedule is considered serializable only if it is equivalent to some serial schedule.

Types of Serializable Schedules:

1. Conflict Serializable
2. View Serializable

Conflict Serializability-

A schedule is Conflict Serializable if it can be converted into a serial schedule by swapping non-conflicting operations.

Conditions for Conflict

Two operations are said to be conflicting if:

1. They belong to different transactions.

2. They access the same data item.
3. At least one operation is a write.

If all three conditions are satisfied, the operations are conflicting.

Examples of Conflicting Operations

Operations	Conflict?	Reason
W(X) of T1 and R(X) of T2	Yes	Same item, different transactions, one write
W(X) of T1 and W(X) of T2	Yes	Same item, different transactions, both write
W(X) of T1 and W(Y) of T2	No	Different data items
R(X) of T1 and R(X) of T2	No	No write operation
W(X) and R(X) of T1	No	Same transaction

Characteristics:

- Guarantees conflict serializability.
- Easy to verify using a precedence graph.
- Every conflict serializable schedule is also serializable.

View Serializability-

A schedule is View Serializable if it is view equivalent to some serial schedule.

If a schedule is conflict serializable, then it is always view serializable.

If a schedule is not conflict serializable, we check for blind write.

- If blind write exists and the schedule is view equivalent to a serial schedule, then it is view serializable.
- If there is no blind write, then the schedule is not view serializable.

Blind Write

A Blind Write occurs when a transaction performs a write operation without first reading the data item.

Example:

Write(X)

without executing

Read(X)

Conditions for View Equivalence

Two schedules are View Equivalent if they satisfy all the following conditions:

1. Initial Read

The same transaction should perform the first read of every data item in both schedules.

2. Final Write

The last write on every data item must be performed by the same transaction in both schedules.

3. Update Read

If a transaction reads a value written by another transaction in one schedule, it should read the value written by the same transaction in the other schedule also.

Difference Between Conflict and View Serializability

Conflict Serializability	View Serializability
Based on conflicting operations.	Based on view equivalence.
Easy to check using precedence graph.	Difficult to check.
Swapping non-conflicting operations is allowed.	Based on Initial Read, Final Write, and Update Read conditions.
Every conflict serializable schedule is view serializable.	Not every view serializable schedule is conflict serializable.
Does not require blind write.	Blind write may be required.

Summary

Concept	Description
Schedule	Sequence of transaction operations.
Serial Schedule	One transaction executes completely before another starts.
Non-Serial Schedule	Transactions execute concurrently.
Serializability	Non-serial schedule behaves like a serial schedule.
Conflict Serializable	Can be converted into a serial schedule by swapping non-conflicting operations.
View Serializable	View equivalent to a serial schedule.

5.Explain the Precedence Graph Method to Determine Conflict Serializability with an Example.

Precedence Graph (Serialization Graph)

A Precedence Graph (also called a Serialization Graph) is a directed graph used to determine whether a schedule is Conflict Serializable.

- Each transaction is represented as a node (vertex).
- A directed edge represents a conflict between two transactions.
- If the graph contains no cycle, the schedule is Conflict Serializable.
- If the graph contains a cycle (loop), the schedule is Not Conflict Serializable.

Steps to Construct a Precedence Graph:

Step 1:

Draw one node for each transaction.

Example:

T1 T2 T3

Step 2:

Identify all conflicting operations.

Two operations conflict if:

- They belong to different transactions.
- They access the same data item.
- At least one operation is a write.

Conflicting pairs are:

- Read – Write (R–W)
- Write – Read (W–R)
- Write – Write (W–W)

Step 3:

For every conflicting pair:

- If an operation of T_i occurs before T_j , draw an edge

$T_i \rightarrow T_j$

Step 4:

Repeat this for all operations in the schedule.

Step 5:

After constructing the graph, check for cycles.

- No Cycle $\rightarrow$ Conflict Serializable
- Cycle Present $\rightarrow$ Not Conflict Serializable

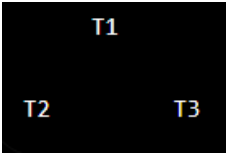
Example-

Consider the following schedule:

Time	T1	T2	T3
1	R(X)		
2			R(Y)
3			R(X)
4		R(Y)	
5		R(Z)	
6			W(Y)
7		W(Z)	
8	R(Z)		
9	W(X)		
10	W(Z)		

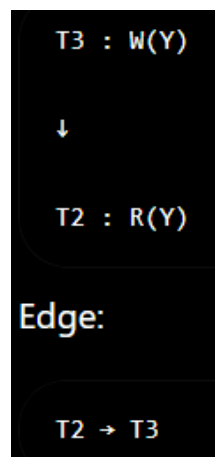
This is the same example discussed in the PPT.

Step 1: Draw Nodes



Step 2: Find Conflicting Operations

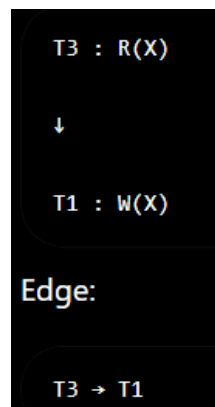
Conflict 1:



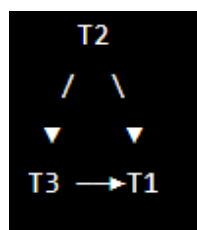
Conflict 2:



Conflict 3:



Step 3: Draw the Graph:



Edges:

- T2 → T3
- T2 → T1

- $T3 \rightarrow T1$

Step 4: Check for Cycle

The graph contains no cycle.

Therefore, The schedule is Conflict Serializable.

Determining the Serializable Order

To determine the equivalent serial schedule:

Rule-

Choose the node having indegree = 0.

- Remove that node.
- Repeat until all nodes are removed.

Step 1:

$\text{Indegree}(T2)=0$

Remove T2

Remaining:

$T3 \rightarrow T1$

Step 2

Now, $\text{Indegree}(T3)=0$

Remove T3

Remaining:

T1

Step 3

Finally,

Remove T1

Hence, Serializable Order

$T2 \rightarrow T3 \rightarrow T1$

This is the serial schedule equivalent to the given non-serial schedule.

Algorithm for Precedence Graph

1. Draw one node for each transaction.
2. Identify all conflicting operations (R-W, W-R, W-W).
3. Draw directed edges according to the order of conflicting operations.
4. Repeat for all transactions.
5. Check for cycles.
6. If there is no cycle, the schedule is Conflict Serializable.
7. If a cycle exists, the schedule is Not Conflict Serializable.
8. Determine the serial order using indegree = 0.

Advantages of Precedence Graph

- Easy method to test conflict serializability.
- Detects dependency among transactions.
- Ensures database consistency.
- Helps determine the equivalent serial schedule.

6.What is Recoverability? Explain Recoverable, Cascading, Cascadeless and Strict Schedules with Suitable Examples.

Recoverability

Recoverability is the ability of a DBMS to recover the database to a consistent state after a transaction failure.

A schedule is said to be recoverable if a transaction commits only after the transaction from which it has read data has committed successfully.

Recoverability ensures that committed transactions never depend on data written by aborted transactions.

Types of Schedules

Recoverability is classified into:

1. Irrecoverable Schedule
2. Recoverable Schedule
3. Cascading Schedule (Cascading Rollback)
4. Cascadeless Schedule
5. Strict Schedule

1. Irrecoverable (Non-Recoverable) Schedule:

An Irrecoverable Schedule is one in which a transaction reads uncommitted data from another transaction and commits before that transaction commits.

If the first transaction later aborts, the committed transaction cannot be recovered.

Example

T1	T2
Read(A)	
Write(A)	
	Read(A) (<i>Dirty Read</i>)
	Write(A)
	Commit
Abort	

Explanation-

- T2 reads the value written by T1 before T1 commits.
- T2 commits.
- Later, T1 aborts.
- Now T2 has committed using incorrect data.

Hence, the schedule is Irrecoverable.

2. Recoverable Schedule:

A Recoverable Schedule is one in which a transaction reads uncommitted data, but delays its commit until the transaction from which it read either commits or rolls back.

Example-

T1	T2
Read(A)	
Write(A)	
	Read(A)
Commit	
	Commit

Explanation-

- T2 reads data written by T1.
- T2 does not commit immediately.
- T1 commits first.
- Then T2 commits.

If T1 had aborted, T2 could also be rolled back.

Hence, the schedule is Recoverable.

3. Cascading Schedule (Cascading Rollback):

A Cascading Schedule is a recoverable schedule in which one transaction failure causes a series of rollbacks in other dependent transactions.

Example-

T10 → T11 → T12

Suppose:

- T11 reads data written by T10.
- T12 reads data written by T11.

If T10 fails,

then

- T11 must roll back.
- T12 must also roll back.

This chain of rollbacks is called Cascading Rollback.

Disadvantages

- Wastes CPU time.
- Many transactions are rolled back.
- Reduces system performance.

Why are Cascading Rollbacks Undesirable?

Cascading rollbacks are undesirable because:

- A single transaction failure causes multiple rollbacks.
- Large amounts of completed work are lost.
- CPU utilization decreases.
- System performance is degraded.
- Recovery becomes more complex.

4. Cascadeless Schedule

A Cascadeless Schedule is one in which a transaction cannot read data written by another transaction until that transaction has committed or aborted.

In simple words, You can write, but you cannot read uncommitted data.

Example-

T1	T2	T3
Write(A)		
Commit		
	Read(A)	
	Commit	
		Read(A)
		Commit

Explanation:

- T2 reads A only after T1 commits.
- T3 reads A only after T2 commits.
- No dirty reads occur.
- Therefore, no cascading rollback occurs.

Advantages:

- Prevents dirty reads.
- No cascading rollbacks.
- Saves CPU time.
- Achieved using locks.

5. Strict Schedule:

A Strict Schedule is one in which a transaction is not allowed to read or write a data item until the transaction that last wrote it has committed or aborted.

Thus, neither reading nor writing is permitted on uncommitted data.

Example:

T1	T2
Write(A)	
Commit	
	Read(A)
	Write(A)

Here,

- T2 cannot read A before T1 commits.
- T2 cannot write A before T1 commits.

Hence, the schedule is Strict.

Characteristics-

- Prevents dirty reads.
- Prevents dirty writes.
- Ensures database consistency.
- Usually less efficient because transactions wait longer.

Difference Between Recoverable, Cascadeless and Strict Schedules

Recoverable Schedule	Cascadeless Schedule	Strict Schedule
Commit is delayed until the transaction from which data is read commits.	Reading uncommitted data is not allowed.	Neither reading nor writing uncommitted data is allowed.
Dirty reads may occur.	Dirty reads are prevented.	Dirty reads and dirty writes are prevented.
Cascading rollback may occur.	Cascading rollback does not occur.	Cascading rollback does not occur.
Less strict.	More strict than Recoverable.	Most strict and safest.

Summary:

Schedule	Description
Irrecoverable	Transaction commits after reading uncommitted data; cannot recover if the writer aborts.
Recoverable	Commit is delayed until the writer transaction commits.
Cascading	One failure causes multiple rollbacks.
Cascadeless	Reading uncommitted data is not allowed.
Strict	Neither reading nor writing uncommitted data is allowed.

7.Explain Pessimistic and Optimistic Concurrency Control Schemes. Compare Both Approaches.

Concurrency Control Schemes:

Concurrency control is the management of simultaneous access to database resources by multiple transactions while maintaining database consistency.

There are two main concurrency control schemes:

1. Pessimistic Concurrency Control
2. Optimistic Concurrency Control

1. Pessimistic Concurrency Control:

A Pessimistic Concurrency Control scheme assumes that conflicts between transactions are likely to occur.

Therefore, it locks the required data item before accessing it and does not release the lock until the transaction is completed.

This prevents other transactions from modifying the same data simultaneously.

Locking Mechanism-

The pessimistic approach uses locks to control concurrent access.

There are two types of locks:

1. Shared Lock (S):

- Used when a transaction only reads data.
- Multiple transactions can hold a shared lock on the same data simultaneously.
- No Exclusive Lock is allowed while a Shared Lock exists.

Example:

Two users checking the balance of the same bank account can both read it at the same time.

2. Exclusive Lock (X):

- Used when a transaction reads and writes data.
- Only one transaction can hold an Exclusive Lock.
- No other Shared or Exclusive Lock is allowed until it is released.

Example:

If one transaction updates an account balance, no other transaction can read or modify that account until the update is complete.

Lock Compatibility Matrix-

Existing Lock	Requested Shared Lock	Requested Exclusive Lock
Shared Lock (S)	✓ Allowed	✗ Not Allowed
Exclusive Lock (X)	✗ Not Allowed	✗ Not Allowed

Locking Protocols Used

The pessimistic approach mainly uses:

- Two-Phase Locking (2PL)
- Timestamp Ordering Protocol

These protocols ensure conflict serializability and maintain database consistency.

Advantages of Pessimistic Concurrency Control:

- Prevents conflicts before they occur.
- Maintains high data consistency.
- Suitable for systems with frequent updates.
- Reduces chances of transaction rollback.

Disadvantages:

- Lower concurrency.
- Transactions may wait for locks.
- Deadlocks may occur.

- Longer waiting time.

2. Optimistic Concurrency Control:

Optimistic Concurrency Control assumes that conflicts are rare.

Transactions execute without locking the data.

The transaction is checked only at the end before committing.

If no conflict is found, the transaction commits; otherwise, it is rolled back.

Phases of Optimistic Concurrency Control

Optimistic Concurrency Control consists of three phases.

Phase 1: Read Phase-

- Transaction reads the required data.
- Performs all calculations on temporary variables.
- Actual database is not modified.

Phase 2: Validation Phase-

- Temporary values are compared with the actual database.
- The DBMS checks whether serializability is violated.

If validation is successful,

→ Proceed to Write Phase.

Otherwise,

→ Transaction is rolled back.

This phase is the most important because it determines whether the transaction can commit.

Phase 3: Write Phase-

- Validated data is written into the database.
- Changes become permanent.

If validation fails, the transaction is rolled back.

Validation in Optimistic Concurrency Control

Validation checks whether the transaction conflicts with any other concurrent transaction.

Three timestamps are maintained:

1. **Start(Ti)** – Time when the transaction starts.
2. **Validation(Ti)** – Time when validation begins.
3. **Finish(Ti)** – Time when the write phase finishes.

The Validation Timestamp is considered the transaction timestamp because it determines whether the transaction commits or rolls back.

Applications of Optimistic Concurrency Control-

Optimistic concurrency control is preferred when:

- Conflicts are very rare.
- Read operations are more frequent than write operations.
- Transactions are short.
- Database contention is low.

Examples:

- Online shopping catalog browsing
- Library management systems
- Reporting systems
- Read-intensive web applications

Difference Between Pessimistic and Optimistic Concurrency Control

Pessimistic Concurrency Control	Optimistic Concurrency Control
Assumes conflicts are likely.	Assumes conflicts are rare.
Uses locks before accessing data.	Does not use locks during execution.
Prevents conflicts before they occur.	Detects conflicts at commit time.
Transactions may wait for locks.	No waiting for locks.
Deadlocks may occur.	No deadlocks due to locking.
Suitable for high-contention systems.	Suitable for low-contention systems.
Lower concurrency.	Higher concurrency.
Fewer rollbacks.	More rollbacks if validation fails.

Summary

Scheme	Main Idea
Pessimistic	Lock first, then execute the transaction.
Optimistic	Execute first, validate later before committing.

8.Explain Locking Mechanisms Used in Pessimistic Concurrency Control. Discuss Shared Lock, Exclusive Lock and Two-Phase Locking (2PL).

Locking Mechanism:

A Lock is a mechanism used in DBMS to control concurrent access to data. It ensures that data integrity and consistency are maintained when multiple transactions execute simultaneously.

Before a transaction reads or writes a data item, it must obtain the appropriate lock. This prevents other transactions from making conflicting changes to the same data item.

Types of Locks-

There are two types of locks:

1. Shared Lock (S)
2. Exclusive Lock (X)

1. Shared Lock (S)-

A Shared Lock (S) is used when a transaction only wants to read a data item.

Multiple transactions can hold a Shared Lock on the same data simultaneously.

However, while a Shared Lock exists, no Exclusive Lock can be granted.

Characteristics:

- Used for Read operation.
- Multiple transactions can read simultaneously.
- Prevents modification while reading.

Example:

Suppose two users want to check Steve's account balance.

T1 → Read(Account)

T2 → Read(Account)

Both transactions are allowed because both require only Shared Locks.

2. Exclusive Lock (X)-

An Exclusive Lock (X) is used when a transaction wants to read and modify a data item.

Only one transaction can hold an Exclusive Lock at a time.

While an Exclusive Lock exists,

- No Shared Lock is allowed.
- No other Exclusive Lock is allowed.

Characteristics:

- Used for Read and Write operations.
- Only one transaction can access the data.
- Prevents both reading and writing by other transactions.

Example-

Suppose a transaction updates Steve's account balance.

T1

Read(Account)

Update Balance

Write(Account)

While T1 holds the Exclusive Lock,

- T2 cannot read the account.
- T3 cannot update the account.

They must wait until T1 releases the lock.

Lock Compatibility Matrix

Existing Lock	Shared Lock Request	Exclusive Lock Request
Shared Lock (S)	✓ Allowed	✗ Not Allowed
Exclusive Lock (X)	✗ Not Allowed	✗ Not Allowed

Interpretation:

- Multiple Shared Locks are compatible.

- **Exclusive Lock is incompatible with every other lock.**

Locking Rules-

For a transaction to be well-formed:

1. **Lock the data item before reading or writing.**
2. **Do not lock an already locked item.**
3. **Do not unlock an already unlocked item.**
4. **Follow a locking protocol.**

These rules help maintain serializability and database consistency.

Two-Phase Locking Protocol (2PL):

Two-Phase Locking (2PL) is a locking protocol that guarantees Conflict Serializability.

In this protocol, a transaction is divided into two phases:

1. **Growing Phase**
2. **Shrinking Phase**

A transaction cannot obtain new locks after releasing its first lock.

Phases of 2PL-

1. Growing Phase

During this phase,

- **Transaction acquires locks.**
- **No locks are released.**

Characteristics-

- **Only locking is allowed.**
- **Unlocking is not allowed.**

2. Shrinking Phase

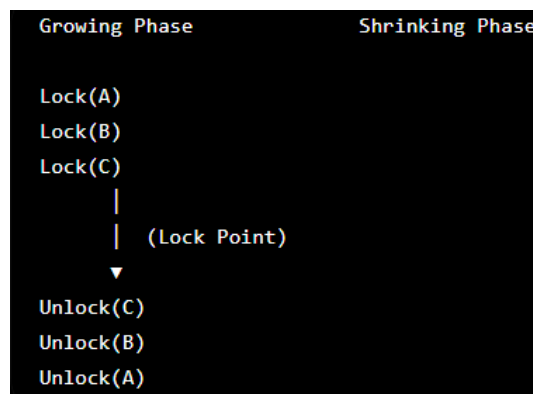
During this phase,

- **Transaction releases locks.**
- **No new locks can be acquired.**

Characteristics-

- **Only unlocking is allowed.**
- **Locking is not allowed.**

Diagram of Two-Phase Locking



Lock Point: The point at which the transaction acquires its last lock.

Properties of 2PL:

- Ensures Conflict Serializability.
- Divides execution into Growing and Shrinking phases.
- Transactions are serialized according to their Lock Point.
- Does not prevent deadlocks.

Advantages of Two-Phase Locking:

- Guarantees conflict serializability.
- Maintains database consistency.
- Prevents concurrent update conflicts.
- Simple locking protocol.

Disadvantages:

- Deadlocks may occur.
- Transactions may wait for locks.
- Reduced concurrency.
- Increased execution time due to waiting.

Summary

Concept	Description
Shared Lock (S)	Used only for reading; multiple transactions can share it.
Exclusive Lock (X)	Used for reading and writing; only one transaction can hold it.
Growing Phase	Locks are acquired; no locks are released.
Shrinking Phase	Locks are released; no new locks are acquired.
Lock Point	Point where the last lock is acquired.

9.explain Deadlocks in DBMS. Discuss Deadlock Prevention and Detection Techniques.

Deadlock

A Deadlock is a condition in which two or more transactions wait indefinitely for one another to release locks, so none of them can proceed.

It is one of the most serious problems in DBMS because all the involved transactions remain in the waiting state until the deadlock is resolved.

Example of Deadlock-

Suppose there are two tables:

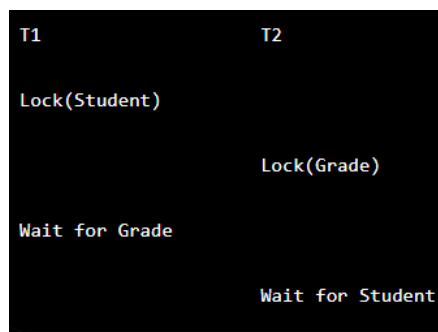
- Student Table
- Grade Table

Transaction T1

- Holds a lock on the Student table.
- Wants to update the Grade table.

Transaction T2

- Holds a lock on the Grade table.
- Wants to update the Student table.



Now,

- T1 waits for T2.
- T2 waits for T1.

Neither transaction can continue.

Hence, a Deadlock occurs.

Deadlock Detection:

Deadlock Detection is the process of identifying whether a deadlock has occurred.

When a transaction waits indefinitely for a lock, the DBMS checks whether it is involved in a deadlock.

The Lock Manager maintains a Wait-for Graph to detect deadlocks.

Wait-for Graph:

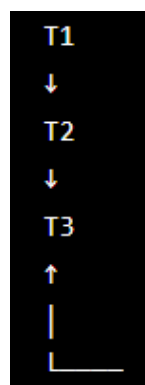
A Wait-for Graph is a directed graph used to detect deadlocks.

- Each transaction is represented by a node.
- A directed edge from $T_i \rightarrow T_j$ means T_i is waiting for T_j to release a lock.

Rule

- Cycle Present $\rightarrow$ Deadlock Exists
- No Cycle $\rightarrow$ No Deadlock

Example



Since a cycle exists, Deadlock is present.

Deadlock Avoidance:

Deadlock Avoidance prevents the database from entering a deadlock state.

Instead of allowing deadlocks to occur and then resolving them, the DBMS checks in advance whether a transaction may cause a deadlock.

If there is a possibility of deadlock, the transaction is delayed or rejected.

This avoids unnecessary rollbacks and saves system resources.

Deadlock Prevention:

Deadlock Prevention ensures that deadlocks never occur by carefully controlling the allocation of resources.

The DBMS analyzes each transaction before execution.

If executing a transaction may lead to a deadlock, it prevents that transaction from proceeding.

Deadlock Prevention Techniques

The PPT discusses two important techniques:

1. Wait-Die Scheme
2. Wound-Wait Scheme

1. Wait-Die Scheme-

The Wait-Die Scheme is based on the timestamps of transactions.

It decides whether a transaction should wait or abort.

Rules:

Suppose:

- T_i requests a resource held by T_j .

Case 1-

If T_i is older than T_j

$TS(T_i) < TS(T_j)$

→ T_i is allowed to wait.

Case 2-

If T_i is younger than T_j

$TS(T_i) > TS(T_j)$

→ T_i is aborted (dies) and restarted later with the same timestamp.

Characteristics:

- Older transaction waits.
- Younger transaction dies and restarts later.

2. Wound-Wait Scheme:

The Wound-Wait Scheme is also based on timestamps.

Rules:

Suppose:

- T_i requests a resource held by T_j .

Case 1-

If T_i is older

$TS(T_i) < TS(T_j)$

→ T_i forces T_j to abort.

Older transaction continues execution.

Case 2-

If T_i is younger

$TS(T_i) > TS(T_j)$

→ T_i waits until the older transaction releases the lock.

Characteristics:

- Older transaction wounds (aborts) younger transaction.
- Younger transaction waits.

Difference Between Wait-Die and Wound-Wait

Wait-Die Scheme	Wound-Wait Scheme
Older transaction waits.	Older transaction aborts younger transaction.
Younger transaction is aborted.	Younger transaction waits if older holds the resource.
Non-preemptive technique.	Preemptive technique.
Older waits, younger dies.	Older wounds, younger waits.

Summary

Technique	Description
Deadlock	Transactions wait indefinitely for each other.
Deadlock Detection	Uses Wait-for Graph to detect cycles.
Deadlock Avoidance	Prevents the system from entering a deadlock state.
Deadlock Prevention	Prevents deadlocks by controlling resource allocation.
Wait-Die	Older waits, younger aborts.
Wound-Wait	Older aborts younger, younger waits for older.

10.Explain the Timestamp Ordering Protocol Used for Concurrency Control.

Timestamp-Based Protocol:

The Timestamp-Based Protocol is a concurrency control protocol that uses timestamps to determine the order of execution of transactions.

Unlike lock-based protocols, it does not use locks. Instead, every transaction is assigned a unique timestamp before it starts execution. The DBMS ensures that conflicting operations are executed according to the timestamp order, thereby maintaining serializability.

Timestamp-

A Timestamp (TS) is a unique number assigned to each transaction before execution begins.

If:

$$TS(T1) < TS(T2)$$

then T1 is older than T2, and the system ensures that T1 appears before T2 in the equivalent serial schedule.

Methods of Assigning Timestamps:

1. System Clock-

- Uses the current system time as the timestamp.
- Every new transaction receives the current clock value.

2. Logical Counter-

- Uses a counter maintained by the DBMS.
- The counter is incremented whenever a new transaction starts.
- The incremented value becomes the transaction's timestamp.

Timestamp Values Maintained for Each Data Item:

For every data item Q, the DBMS maintains two timestamp values.

1. Read Timestamp (R-TS(Q))

- Stores the largest timestamp of any transaction that has successfully read data item Q.

2. Write Timestamp (W-TS(Q))

- Stores the largest timestamp of any transaction that has successfully written data item Q.

These values are updated whenever a read or write operation is successfully executed.

Timestamp Ordering Protocol:

The protocol ensures that all conflicting operations occur in timestamp order.

There are two cases:

Case 1: Read Operation-

Suppose transaction T_i performs:

Read(Q)

Rule 1:

If $TS(T_i) < W-TS(Q)$

then

- The required value has already been overwritten by a newer transaction.
- Read operation is rejected.
- Transaction T_i is rolled back.

Rule 2:

Otherwise,

- Read operation is executed successfully.
- Update

$R-TS(Q) = \text{MAX}(R-TS(Q), TS(T_i))$

Case 2: Write Operation-

Suppose transaction T_i performs:

Write(Q)

Rule 1

If $TS(T_i) < R-TS(Q)$

then

- A newer transaction has already read the data.
- Write operation is rejected.
- Transaction is rolled back.

Rule 2-

If $TS(T_i) < W-TS(Q)$

then

- Transaction is trying to write an obsolete value.
- Write operation is rejected.
- Transaction is rolled back.

Rule 3-

Otherwise,

- Write operation is executed.
- Update

$$W-TS(Q) = TS(T_i)$$

Rollback

If a transaction is rolled back because of any read or write conflict,

- The DBMS assigns a new timestamp.
- The transaction is restarted.

This ensures that timestamp ordering is maintained.

Example

Suppose:

$$TS(T_1) = 100$$

$$TS(T_2) = 200$$

$$TS(T_3) = 300$$

Initially,

$$R-TS(A) = 0$$

$$W-TS(A) = 0$$

Step 1:

T1 performs

Read(A)

Since $100 < 0$ is false,

Read operation succeeds.

Update $R-TS(A) = 100$

Step 2:

T3 performs

Write(A)

Conditions are satisfied.

Update $W-TS(A) = 300$

Step 3:

Now T2 performs Write(A) Since $TS(T2)=200$

$W-TS(A)=300$

Condition $200 < 300$ is True.

Therefore,

- Write operation is rejected.
- T2 is rolled back.

This example illustrates how the Timestamp Ordering Protocol maintains the correct serial order.

Advantages:

- Provides high concurrency.
- Efficient and scalable.
- No locking is required.
- Deadlocks do not occur.
- Improves overall system performance by allowing concurrent execution.

Disadvantages:

- Limited timestamp granularity.
- Proper timestamp management is required.
- Clock synchronization is necessary.
- Assigning unique timestamps may be difficult in distributed systems.

Summary

Concept	Description
Timestamp	Unique number assigned to every transaction.
R-TS(Q)	Largest timestamp of any transaction that has successfully read Q.
W-TS(Q)	Largest timestamp of any transaction that has successfully written Q.
Read Rule	If $TS(T_i) < W-TS(Q)$, rollback; otherwise update R-TS(Q).
Write Rule	If $TS(T_i) < R-TS(Q)$ or $TS(T_i) < W-TS(Q)$, rollback; otherwise update W-TS(Q).

11.Explain the Shadow Paging Recovery Technique with Algorithm, Advantages and Disadvantages.

Shadow Paging

Shadow Paging is a database recovery technique that allows recovery from transaction failures without using a log (in a single-user environment).

It considers the database as a collection of fixed-size disk pages (blocks). Before a transaction begins, a copy of the current page directory called the Shadow Directory is created. During transaction execution, only the Current Directory is modified, while the Shadow Directory remains unchanged. If a failure occurs, the database is restored using the Shadow Directory.

Working of Shadow Paging:

The Shadow Paging technique maintains two directories:

1. Current Directory
2. Shadow Directory

Current Directory-

- Contains pointers to the latest database pages.
- Modified during transaction execution.

Shadow Directory-

- Copy of the Current Directory before the transaction starts.
- Stored safely on disk.
- Never modified during transaction execution.
- Used for recovery if the transaction fails.

Working Procedure:

Step 1-

Initially, the Current Directory points to the latest database pages.

Step 2-

When a transaction starts,

- A copy of the Current Directory is created.
- This copy becomes the Shadow Directory.

Step 3-

Whenever a page is modified,

- A new copy of that page is created.
- The old page is not overwritten.
- The Current Directory is updated to point to the new page.
- The Shadow Directory still points to the original page.

Step 4-

If the transaction commits successfully,

- Shadow Directory is discarded.
- Old pages are deleted.
- New pages become permanent.

Step 5-

If the transaction fails,

- Current Directory is discarded.
- Modified pages are deleted.
- Shadow Directory is restored.
- Database returns to its previous consistent state.

Shadow Paging Algorithm:

Step 1-

Construct a directory in which every entry points to a database block.

Step 2-

Create a Shadow Directory by copying the Current Directory and store it on disk.

Step 3-

During execution,

- Modify only the pages pointed to by the Current Directory.
- Store modified pages at new locations on the disk.

Step 4-

If the transaction commits,

- Discard the Shadow Directory.
- Delete the old pages.

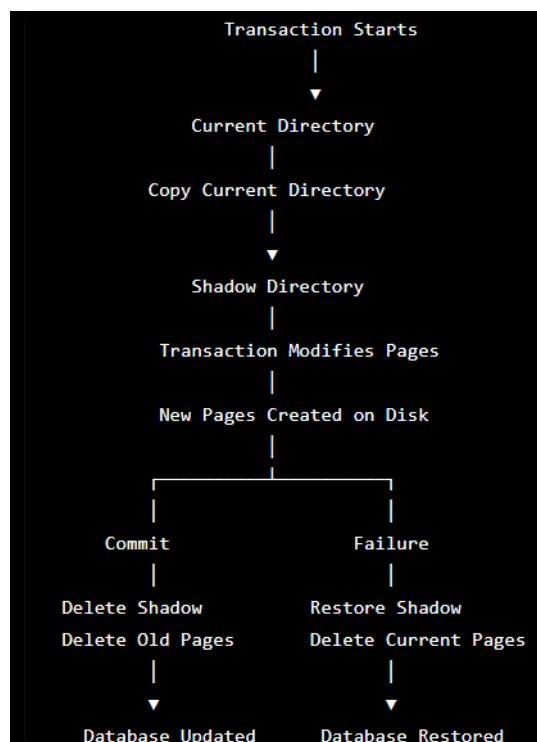
Step 5-

If the transaction fails,

- Discard the Current Directory.
- Delete the modified pages.
- Restore the Shadow Directory.

Since recovery requires neither UNDO nor REDO operations, Shadow Paging is called a NO-UNDO / NO-REDO Recovery Technique.

Diagram of Shadow Paging:



Advantages of Shadow Paging:

1. No log file is required in a single-user environment.
2. Recovery is simple and fast.
3. No UNDO operation is needed.
4. No REDO operation is needed.
5. Database can be restored easily after failure.

Disadvantages of Shadow Paging:

1. Updated pages are stored at new disk locations, making storage management difficult.
2. Writing large Shadow Directories to disk increases overhead.
3. Garbage collection is required to free old pages after commit.
4. Switching between the Current and Shadow Directories must be performed atomically.

Summary-

Concept	Description
Current Directory	Points to the latest database pages.
Shadow Directory	Backup copy of the Current Directory used for recovery.
Commit	Shadow Directory discarded; new pages become permanent.
Failure	Current Directory discarded; Shadow Directory restored.
Recovery Type	NO-UNDO / NO-REDO

12.Explain Parallel Database.

Parallel Database:

A Parallel Database is a database system that uses multiple processors (CPUs) and disks working simultaneously (in parallel) to perform database operations. It improves system performance by executing multiple tasks concurrently.

A parallel database system seeks to improve performance through the parallelization of operations such as loading data, building indexes, and query processing.

Need for Parallel Database:

Nowadays, organizations handle huge volumes of data and require high processing speed.

Traditional centralized or client-server databases become inefficient for such large workloads.

Therefore, parallel databases are used to:

- **Process large amounts of data quickly.**
- **Improve system performance.**
- **Handle many transactions simultaneously.**

Working of Parallel Database:

Step 1-

A large task is divided into several smaller tasks.

Step 2-

These smaller tasks are executed simultaneously using multiple CPUs and disks.

Step 3-

After all tasks are completed, the results are combined to produce the final output.

Unlike serial processing, where tasks execute one after another, parallel processing executes multiple operations at the same time, reducing execution time.

Performance Measures:

1. Throughput-

- **Number of transactions completed in a given period.**
- **Parallel execution increases throughput by processing many transactions simultaneously.**

2. Response Time-

- **Time taken to complete a single transaction.**
- **Parallel execution reduces response time by dividing the work among multiple processors.**

Advantages of Parallel Database:

1. Performance Improvement-

- **Multiple CPUs and disks work together.**
- **Significantly increases processing speed.**

2. High Availability-

- **Failure of one node does not stop the entire database.**

- Other nodes continue processing.

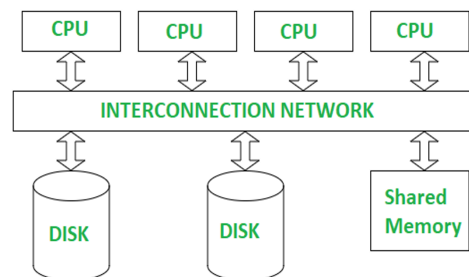
3. Proper Resource Utilization-

- CPUs remain busy because multiple tasks execute simultaneously.
- Better utilization of hardware resources.

4. Increased Reliability-

- If one site fails, another site containing a copy of the data can continue processing.
- Improves system reliability.

Diagram of Parallel Database:



Applications:

- Large banking systems
- Airline reservation systems
- Scientific research
- Data warehouses
- E-commerce websites

Summary-

Feature	Description
Definition	Database using multiple processors and disks simultaneously.
Main Purpose	Improve speed and performance.
Performance Measures	Throughput and Response Time.
Advantages	High performance, availability, reliability, efficient resource utilization.

13.Explain Multimedia Database.

Multimedia Database

A Multimedia Database is a collection of interrelated multimedia data such as text, graphics, images, animations, audio, and video.

The system that stores, manages, retrieves, and delivers different types of multimedia data is called a Multimedia Database Management System (MMDBMS).

Types of Multimedia Data:

A Multimedia Database contains different types of media.

1. Static Media-

- Media that does not change with time.
- Example:
 - Images
 - Text
 - Graphics

2. Dynamic Media-

- Media that changes over time.
- Example:
 - Audio
 - Video
 - Animation

3. Dimensional Media-

- Media having three-dimensional representation.
- Example:
 - 3D models
 - CAD designs

Contents of Multimedia Database:

A Multimedia Database stores four types of information.

1. Media Data-

Actual multimedia content.

Examples:

- Image
- Video
- Audio file

2. Media Format Data-

Information describing the format of multimedia data.

Examples include:

- Sampling rate
- Resolution
- Encoding scheme
- JPEG
- MP3
- MP4

3. Media Keyword Data-

Descriptive information about the multimedia content.

Examples:

- Date
- Time
- Place of recording
- Keywords

4. Media Feature Data-

Information extracted from the content itself.

Examples:

- Color distribution
- Texture
- Shape
- Object features

Summary of Multimedia Database Contents

Type of Data	Meaning	Example
Media Data	Actual multimedia content	Photo, Video, Audio
Media Format Data	Format information	JPEG, MP3, MP4
Media Keyword Data	Descriptive information	Date, Time, Location
Media Feature Data	Content characteristics	Color, Shape, Texture

Types of Multimedia Applications:

1. Repository Applications-

These applications store a large amount of multimedia data along with metadata for future retrieval.

Examples

- Satellite image database
- MRI and X-ray storage
- Engineering drawing archive

2. Presentation Applications-

These applications deliver multimedia smoothly in real time.

They require proper timing and synchronization.

Examples

- Video editing
- Live streaming
- Online lectures

3. Collaborative Multimedia Applications-

These applications allow multiple users to work together using multimedia data.

Examples

- Healthcare systems
- Engineering design collaboration
- Online team projects

Applications of Multimedia Database:

Multimedia Databases are widely used in:

1. Documents and Record Management-

Example:

- Insurance claim records

2. Knowledge Dissemination-

Example:

- Electronic books (E-books)

3. Education and Training-

Example:

- Digital libraries
- Computer-aided learning

4. Marketing, Advertising and Entertainment-

Example:

- Virtual city tours
- Online advertisements

5. Real-Time Monitoring and Control-

Example:

- Manufacturing operation control

Challenges of Multimedia Database:

1. Modelling

- Difficult to integrate database and information retrieval techniques.

2. Design

- Complex due to different multimedia formats such as JPEG, GIF, PNG, MPEG.

3. Storage

- Requires large storage space.
- Needs efficient compression and buffering.

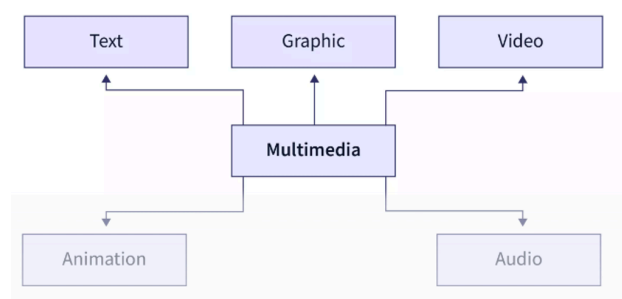
4. Performance

- Audio and video processing consume high processing time and bandwidth.

5. Query Processing and Retrieval

- Efficient searching and retrieval of multimedia content is difficult.

Diagram of Multimedia Database



Advantages:

- Stores different types of multimedia data.
- Easy retrieval of multimedia information.
- Supports education, healthcare, entertainment, and business applications.
- Provides efficient multimedia management.

Summary-

Feature	Description
Definition	Database storing multimedia data such as text, images, audio, and video.
Contents	Media Data, Format Data, Keyword Data, Feature Data.
Applications	Education, Healthcare, Entertainment, Marketing, Digital Libraries.
Challenges	Storage, Design, Performance, Query Processing.

14.Explain Mobile Database.

Mobile Database-

A Mobile Database is a database that can be accessed by a mobile computing device through a mobile (wireless) network.

In a mobile database system, both the client and the server communicate through wireless connections. Mobile databases can be used on devices such as Android phones, iOS devices, tablets, and laptops.

Features of Mobile Database:

The important features of a Mobile Database are:

1. Cache Maintenance

- A cache is maintained to store frequently accessed transactions.
- Prevents loss of data during connection failures.

2. Physically Separate Database

- Mobile databases are physically separate from the central database server.

3. Resides on Mobile Devices

- The database resides on devices such as smartphones and tablets.

4. Wireless Communication

- Mobile databases communicate with the central database server or other mobile devices through wireless networks.

5. Offline Access

- Mobile users can continue working even when there is no wireless connection.

6. Data Analysis

- Mobile databases allow users to analyze and manipulate data directly on mobile devices.

Components of Mobile Database:

A Mobile Database typically consists of three parties.

1. Fixed Hosts-

- Perform transaction processing and data management.
- Use database servers to store and manage data.

2. Mobile Units-

- Portable computing devices.
- Communicate with base stations through wireless networks.

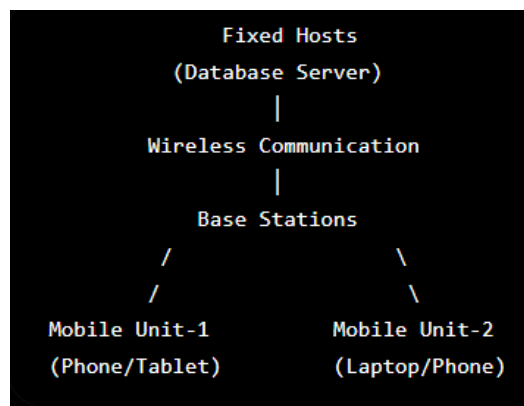
Examples

- Smartphones
- Tablets
- Laptops

3. Base Stations-

- Fixed wireless communication stations.
- Transfer data between mobile units and fixed hosts.

Architecture of Mobile Database-



Advantages of Mobile Database:

- Provides access to data from anywhere.
- Supports wireless communication.
- Allows users to work even during network disconnection.
- Improves flexibility and mobility.
- Enables real-time access to information.

Limitations of Mobile Database:

1. Limited Wireless Bandwidth

- Data transfer speed is limited.

2. High Battery Requirement

- Mobile devices require sufficient battery power for continuous access.

3. Less Secure

- Wireless communication is more vulnerable to security threats.

4. Theft Risk

- Mobile devices are easier to lose or steal.

Applications of Mobile Database:

- Mobile Banking
- Online Shopping
- Mobile Healthcare
- GPS and Navigation Systems
- Mobile Ticket Booking
- Food Delivery Applications

Summary

Feature	Description
Definition	Database accessed through wireless/mobile networks.
Main Components	Fixed Hosts, Mobile Units, Base Stations.
Features	Cache support, wireless communication, offline access.
Limitations	Limited bandwidth, battery dependence, security issues, theft risk.

15.Explain Web Database.

Web Database

A Web Database is a database system that stores and displays information which can be accessed through the Internet or a web browser.

It is used to store information such as membership details, customer records, inventory, e-commerce data, blogs, e-mails, and other online applications.

A Web Database is usually hosted on websites and accessed using a web browser. It is also known as a Web-Based Database Management System (Web-DBMS).

Need for Web Database-

A Web Database is useful when:

- Information needs to be shared among multiple users.
- Data must be accessed from different locations.
- Multiple devices such as computers, tablets, and smartphones need access to the same database.
- Online applications require centralized data storage.

Working of Web Database-

1. Data is stored on a web server.
2. Users access the database using a web browser.
3. The browser sends a request to the server.
4. The server retrieves the required information from the database.
5. The requested data is displayed on the user's browser.

Example:

Online Shopping Website

Suppose a customer searches for:

"Black Dress"

The website searches its database and displays all products containing the keywords "Black" and "Dress".

This information is retrieved from the Web Database.

Features of Web Database:

1. Simplicity

- Easy to access and manage through a web browser.

2. Platform Independence

- Works on different operating systems and devices.

3. Graphical User Interface (GUI)

- Provides a user-friendly interface.

4. Standardization

- Supports standard web technologies.

5. Cross-Platform Support

- Can be accessed from computers, tablets, and mobile phones.

6. Transparent Network Access

- Users can access the database from different locations through the Internet.

7. Scalability

- Can handle increasing numbers of users and large amounts of data.

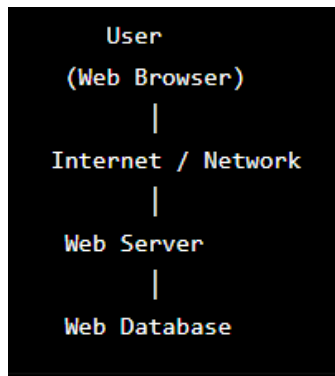
8. Innovation

- Supports modern web-based applications and services.

9. Accessibility

- Information can be accessed from almost any device connected to the Internet.

Diagram of Web Database



Applications of Web Database:

Web Databases are widely used in:

- E-Commerce websites
- Online Banking
- Library Management Systems
- Student Information Systems
- Hospital Management Systems
- Blogs and Email Services
- Online Reservation Systems

Advantages:

- Easy access through web browsers.
- Accessible from anywhere with an Internet connection.
- Supports multiple users simultaneously.
- Platform independent.
- Easy sharing of information.
- Highly scalable.

Summary

Feature	Description
Definition	Database accessible through the Internet using a web browser.
Main Purpose	Store and share online information.
Features	GUI, platform independence, scalability, cross-platform support.
Applications	E-commerce, banking, blogs, email, online services.

16.Explain Data Warehouse.

Data Warehouse-

A Data Warehouse is a centralized repository that stores large volumes of historical data collected from different sources for analysis and decision-making.

It is designed to support reporting, data analysis, and business intelligence, rather than day-to-day transaction processing.

Need for Data Warehouse-

A Data Warehouse is used to:

- Store data collected from multiple sources.
- Analyze historical data.
- Generate reports for management.
- Support business decision-making.
- Improve business planning and forecasting.

Characteristics of Data Warehouse:

1. Subject-Oriented

- Data is organized according to specific business subjects such as customers, sales, or products.

2. Integrated

- Data from different sources is combined into a single repository.

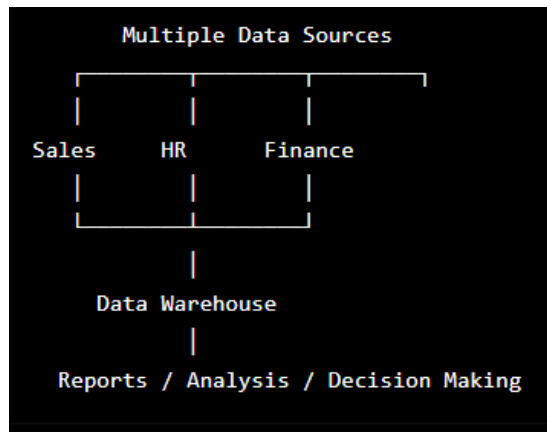
3. Time-Variant

- Stores historical data over a long period.

4. Non-Volatile

- Once data is stored, it is not frequently modified or deleted.

Architecture of Data Warehouse:



Advantages of Data Warehouse:

- Stores large amounts of historical data.
- Supports decision-making.
- Improves query performance.
- Integrates data from multiple sources.
- Generates analytical reports.
- Supports business intelligence applications.

Applications:

Data Warehouses are widely used in:

- Banking
- Healthcare
- Retail
- Insurance
- Educational Institutions
- Government Organizations

Summary

Feature	Description
Definition	Central repository of historical data.
Purpose	Analysis and decision-making.
Data Sources	Multiple heterogeneous sources.
Applications	Banking, Healthcare, Retail, Business Intelligence.

17.Differentiate Between OLTP and OLAP.

OLTP (Online Transaction Processing):

OLTP (Online Transaction Processing) is a database system designed to manage day-to-day business transactions quickly and efficiently.

It is mainly used for insert, update, delete, and retrieval operations performed by multiple users simultaneously.

Examples-

- **ATM Transactions**
- **Online Banking**
- **Railway Reservation System**
- **E-Commerce Order Processing**

OLAP (Online Analytical Processing):

OLAP (Online Analytical Processing) is a database system designed to analyze large volumes of historical data for business intelligence and decision-making.

It is mainly used for complex queries, reporting, trend analysis, and data mining.

Examples-

- **Sales Analysis**
- **Business Intelligence**
- **Market Trend Analysis**
- **Financial Forecasting**

Difference Between OLTP and OLAP:

OLTP	OLAP
Stands for Online Transaction Processing.	Stands for Online Analytical Processing.
Used for day-to-day transaction processing.	Used for data analysis and decision-making.
Handles current operational data.	Handles historical data.
Supports Insert, Update, Delete, and Select operations.	Supports complex analytical queries.
Large number of short transactions.	Fewer but complex queries.
Response time is very fast.	Response time is comparatively slower due to complex analysis.
Data is frequently updated.	Data is mostly read-only and updated periodically.
Used by clerks, customers, and operational staff.	Used by managers, analysts, and executives.
Focuses on transaction processing.	Focuses on decision support and business intelligence.
Examples: Banking, ATM, Railway Reservation, Online Shopping.	Examples: Sales Analysis, Data Warehouse, Business Reports.

Advantages of OLTP-

- **Fast transaction processing.**
- **Supports multiple users simultaneously.**
- **Ensures data consistency.**
- **High availability.**
- **Suitable for real-time applications.**

Advantages of OLAP-

- **Supports business decision-making.**
- **Performs complex data analysis.**
- **Identifies business trends.**
- **Generates reports and summaries.**

- Helps in forecasting and planning.

Applications:

OLTP Applications-

- Banking Systems
- ATM Machines
- Airline Reservation Systems
- Online Shopping Websites
- Hospital Management Systems

OLAP Applications-

- Data Warehouses
- Business Intelligence Systems
- Sales Forecasting
- Financial Analysis
- Marketing Analysis

Summary

Feature	OLTP	OLAP
Purpose	Transaction Processing	Data Analysis
Data	Current Data	Historical Data
Users	Operational Users	Managers & Analysts
Operations	CRUD Operations	Analytical Queries
Speed	Very Fast	Moderate (Complex Queries)

18.Explain NoSQL Database.

NoSQL Database:

NoSQL stands for Not Only SQL.

A NoSQL Database is a non-relational database designed to store and manage large volumes of structured, semi-structured, and unstructured data. Unlike traditional relational databases, NoSQL databases do not use tables with fixed schemas and are designed to provide high scalability, flexibility, and performance.

Need for NoSQL Database-

Traditional Relational Database Management Systems (RDBMS) face limitations when handling:

- Very large volumes of data.
- Unstructured or semi-structured data.
- High-speed web applications.
- Distributed systems.

NoSQL databases overcome these limitations by providing flexible data storage and horizontal scalability.

Features of NoSQL Database:

1. Schema-less Database

- No fixed table structure.
- Different records can have different fields.

2. High Scalability

- Easily handles increasing amounts of data by adding more servers.

3. High Performance

- Provides faster read and write operations for large datasets.

4. Distributed Architecture

- Data is stored across multiple servers.

5. Flexible Data Model

- Supports structured, semi-structured, and unstructured data.

6. High Availability

- Data remains available even if one server fails.

Types of NoSQL Databases:

1. Key-Value Database

- Stores data as key-value pairs.

Example:

Student_ID → Vyshali

2. Document Database

- Stores data in the form of documents such as JSON or XML.

Example:{

"Name":"Vyshali",

"Branch":"CSE"

}

3. Column-Family Database

- Stores data in columns instead of rows.
- Suitable for large-scale data processing.

4. Graph Database

- Stores data as nodes and relationships.
- Used for highly connected data.

Example:

- Social networks
- Recommendation systems

Advantages of NoSQL Database-

- High scalability.
- Faster data access.
- Handles huge volumes of data.
- Flexible schema.
- Supports distributed computing.
- Suitable for cloud applications.
- High availability and fault tolerance.

Disadvantages of NoSQL Database-

- Less support for complex SQL queries.
- Data redundancy may occur.
- Limited support for joins.
- Weaker consistency compared to traditional RDBMS in some systems.
- Not suitable for applications requiring complex transactions.

Applications of NoSQL Database-

NoSQL databases are widely used in:

- Social Media Applications
- Big Data Analytics
- E-commerce Websites

- Content Management Systems
- Real-Time Web Applications
- Cloud Computing
- Internet of Things (IoT)

Difference Between SQL and NoSQL

SQL Database	NoSQL Database
Relational database.	Non-relational database.
Fixed schema.	Flexible (schema-less).
Stores data in tables.	Stores data as key-value, documents, columns, or graphs.
Uses SQL language.	Does not rely only on SQL.
Vertical scalability.	Horizontal scalability.
Best for structured data.	Best for structured, semi-structured, and unstructured data.

Summary

Feature	Description
Definition	Non-relational database for large-scale data storage.
Data Model	Key-Value, Document, Column-Family, Graph.
Main Features	Scalability, Flexibility, High Performance.
Applications	Big Data, Social Media, Cloud Computing, IoT.



SAQ's:

1. Define Transaction.

A transaction is a set of logically related operations performed as a single unit of work in a database.

An application program may contain several transactions separated by Begin Transaction and End Transaction boundaries.

The basic operations of a transaction are:

- **Read_item(X):** Reads a database item into a program variable.
- **Write_item(X):** Writes the updated value back into the database.

Example: Bank money transfer from one account to another.

2. Explain Read and Write Operations.

Read Operation

The read_item(X) operation performs the following steps:

1. Find the address of the disk block containing X.
2. Copy the disk block into the main memory buffer.
3. Copy X from the buffer into the program variable.

Write Operation

The write_item(X) operation performs the following steps:

1. Find the address of the disk block containing X.
2. Copy the disk block into the main memory buffer.
3. Update X in the buffer.
4. Store the updated block back to the disk.

3. Recovery Manager.

The Recovery Manager is responsible for maintaining database consistency after transaction failures.

It performs the following operations:

- **Begin Transaction**
- **Read**
- **Write**
- **End Transaction**
- **Commit**
- **Rollback (Abort)**
- **Undo**
- **Redo**

Its main purpose is to recover the database after failures and maintain consistency.

4. System Log and Log Records.

A System Log (Journal) is a file that stores all transaction activities affecting the database.

It is maintained on disk and helps recover the database after failures.

Types of Log Records

- [Start_Transaction, T] – Transaction starts.
- [Read_Item, T, X] – Item X is read.
- [Write_Item, T, X, Old Value, New Value] – Item X is updated.
- [Commit, T] – Transaction successfully completes.
- [Abort, T] – Transaction is aborted.

5. Causes of Transaction Failure.

A transaction may fail due to:

- Computer/System Crash
- Transaction Errors
- External Exceptions
- Concurrency Control Enforcement
- Disk Failure
- Physical Problems (Power failure, Fire, Theft, etc.)

6. Differentiate between Serial and Non-Serial Schedules.

Serial Schedule	Non-Serial Schedule
Transactions execute one after another.	Transactions execute concurrently.
No interleaving.	Operations are interleaved.
Always consistent.	May become inconsistent.
No concurrency.	High concurrency.

7. Recoverable Schedule.

A Recoverable Schedule is one in which a transaction reads uncommitted data but delays its commit until the transaction from which it has read either commits or aborts.

This ensures that the database can recover if the first transaction fails.

8.Cascading Rollback.

A Cascading Rollback occurs when the failure of one transaction causes several other dependent transactions to roll back.

Disadvantages

- Wastes CPU time.
- Reduces performance.
- Increases recovery overhead.

9.Cascadeless Schedule.

A Cascadeless Schedule does not allow a transaction to read data written by another transaction until that transaction commits.

Advantages

- Prevents dirty reads.
- Eliminates cascading rollback.
- Saves CPU time.

10.Strict Schedule.

A Strict Schedule is one in which a transaction cannot read or write a data item until the transaction that last wrote it has committed or aborted.

Advantages

- Prevents dirty reads.
- Prevents dirty writes.
- Ensures high database consistency.

11.Shared Lock and Exclusive Lock.

Shared Lock (S)

- Used for Read operations.
- Multiple transactions can hold it simultaneously.

Exclusive Lock (X)

- Used for Read and Write operations.
- Only one transaction can hold it.
- No other Shared or Exclusive Lock is allowed.

12.Two-Phase Locking Protocol (2PL).

Two-Phase Locking (2PL) guarantees Conflict Serializability.

It has two phases:

1. Growing Phase – Acquire locks only.
2. Shrinking Phase – Release locks only.

No new lock can be acquired after the first lock is released.

13. Deadlock.

A Deadlock is a situation in which two or more transactions wait indefinitely for each other to release locks.

Solution

- Wait-for Graph
- Wait-Die Scheme
- Wound-Wait Scheme

14. Wait-Die Scheme.

The Wait-Die Scheme is a deadlock prevention technique based on timestamps.

- Older transaction → Waits
- Younger transaction → Aborts (Dies)

It is a non-preemptive scheme.

15. Wound-Wait Scheme.

The Wound-Wait Scheme is also based on timestamps.

- Older transaction → Aborts younger transaction
- Younger transaction → Waits

It is a preemptive scheme.

16. Lock Manager.

The Lock Manager manages all lock and unlock requests.

Its functions are:

- Grants locks if data is available.
- Makes transactions wait if data is locked.
- Detects deadlocks.
- Maintains the Lock Table containing all lock information.

17. Timestamp Ordering Protocol.

The Timestamp Ordering Protocol assigns a unique timestamp to every transaction.

It maintains:

- Read Timestamp (R-TS)
- Write Timestamp (W-TS)

Transactions violating timestamp order are rolled back.

Advantages

- No locks.
- No deadlocks.
- High concurrency.

18.Validation-Based Protocol (Optimistic Concurrency Control).

The Validation-Based Protocol executes transactions in three phases:

1. Read Phase
2. Validation Phase
3. Write Phase

Validation checks whether serializability is violated before committing the transaction.

19.Shadow Paging.

Shadow Paging is a recovery technique that uses:

- Current Directory
- Shadow Directory

If a transaction commits, the Shadow Directory is discarded.

If it fails, the Current Directory is discarded and the Shadow Directory is restored.

It is a NO-UNDO/NO-REDO recovery technique.

20.Parallel Database.

A Parallel Database uses multiple processors and disks simultaneously to improve database performance.

Advantages

- High throughput.
- Reduced response time.
- Better resource utilization.
- High reliability.

21.Multimedia Database.

A Multimedia Database stores multimedia data such as:

- Text
- Images
- Audio
- Video
- Animation

It also stores media format, keyword, and feature data for efficient retrieval.

22.Mobile Database.

A Mobile Database is accessed through wireless networks using mobile devices.

Components

- Fixed Hosts
- Mobile Units
- Base Stations

Advantages

- Mobility.
- Offline access.
- Wireless communication.

23.Web Database.

A Web Database is a database accessible through the Internet using a web browser.

Features

- Platform independent.
- GUI support.
- Cross-platform access.
- Scalability.

24.OLTP vs OLAP.

OLTP	OLAP
Transaction processing	Data analysis
Current data	Historical data
Fast operations	Complex queries
Used by operational users	Used by managers and analysts

25.NoSQL Database.

A NoSQL Database is a non-relational database designed for storing large volumes of structured, semi-structured, and unstructured data.

Features

- Schema-less
- High scalability
- High performance
- Distributed architecture

Applications

- Social Media
- Big Data
- Cloud Computing
- IoT